

Controladores da API Web do ASP.NET Core

Concluído

- 4 minutos

No exercício anterior, você criou um aplicativo Web que fornece dados de previsão do tempo de exemplo e, em seguida, interagiu com ele no REPL (Read-Eval-Print Loop) HTTP.

Antes de você começar a escrever nosso `PizzaController`, vamos dar uma olhada no código do exemplo `WeatherController` para entender como ele funciona. Nesta unidade, você vê como o `WeatherController` usa a classe base `ControllerBase` e alguns atributos .NET para criar uma API Web funcional com algumas dezenas de linhas de código. Depois de entender esses conceitos, você está pronto para escrever sua própria classe `PizzaController`.

Este é o código de toda a classe `WeatherController`. Não se preocupe se ainda não faz sentido. Vamos ver isso passo a passo.

C#

```
using Microsoft.AspNetCore.Mvc;

namespace ContosoPizza.Controllers;

[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
{
    private static readonly string[] Summaries = new[]
    {
        "Freezing", "Bracing", "Chilly", "Cool", "Mild",
        "Warm", "Balmy", "Hot", "Sweltering", "Scorching"
    };

    private readonly ILogger<WeatherForecastController>
        _logger;
```

A classe base: `ControllerBase`

Um controlador é uma classe pública com um ou mais métodos públicos conhecidos como *ações*. Por convenção, um controlador é colocado no diretório *Controllers* da raiz do projeto. As ações são expostas como pontos de extremidade HTTP por meio de roteamento. Portanto, uma solicitação HTTP GET para `https://localhost:{PORT}/weatherforecast` faz com que o método `Get()` da classe `WeatherForecastController` seja executado.

A primeira coisa a notar é que esta classe herda da classe base `ControllerBase`. Essa classe base fornece muitas funcionalidades padrão para lidar com solicitações HTTP, para que você possa se concentrar na lógica de negócios específica do seu aplicativo.

Observação

Se você tiver experiência com Razor Pages ou desenvolvimento de arquitetura de controlador de exibição de modelo (MVC) no ASP.NET Core, você usou a classe `Controller`. Não crie um controlador de API Web derivando da classe base `Controller`. `Controller` deriva de `ControllerBase` e adiciona suporte para exibições, portanto, é para manipular páginas da Web, não solicitações de API Web.

Atributos de classe Controller da API

Dois atributos importantes são aplicados a `WeatherForecastController`, conforme mostrado no código a seguir:

C#

```
[ApiController]
[Route("[controller]")]
public class WeatherForecastController : ControllerBase
```

`[ApiController]` habilita **comportamentos dogmáticos** que facilitam a criação de APIs Web. Alguns comportamentos incluem *inferência de fonte de parâmetro*, *roteamento de atributo como um requisito* e aprimoramentos* *de tratamento de erro de validação de modelo*.

`[Route]` define o padrão de roteamento `[controller]`. O nome do controlador (que não diferencia maiúsculas de minúsculas, sem o sufixo *Controller*) substitui o

token `[controller]` . Esse controlador lida com solicitações para `https://localhost:{PORT}/weatherforecast` .

Observação

A rota pode conter cadeias de caracteres estáticas, como em `api/[controller]` . Neste exemplo, esse controlador lidaria com uma solicitação para `https://localhost:{PORT}/api/weatherforecast` .

Fornecer resultados meteorológicos com o método `Get()`

`WeatherForecastController` inclui uma ação de controlador único, designada pelo atributo `[HttpGet(Name = "GetWeatherForecast")]` . Esse atributo encaminha solicitações HTTP GET para o método `public IEnumerable<WeatherForecast> Get()` . Como você viu no exercício anterior, solicitações para `https://localhost:{PORT}/weatherforecast` fizeram com que os resultados meteorológicos fossem retornados.

Como você aprenderá mais adiante neste módulo, outras ações comuns estão associadas a uma API Web que executa operações CRUD (`GET` , `PUT` , `POST` , `DELETE`). Mas um controlador de API precisa implementar apenas uma ação do controlador.

Neste caso, você está recebendo a lista completa de `WeatherForecast` itens retornados. A operação `GET` também permite recuperar um só item passando um identificador. Em ASP.NET, você pode recuperar um único item usando o atributo `[HttpGet("{id}")]` . Você implementará esse atributo no próximo exercício.

Agora que você conhece os componentes fundamentais de um controlador de API Web, você está pronto para criar a sua própria classe `PizzaController` .